Lab 5: Static Code Analysis

Income Tax Calculator | Consolidated Submission
30 October 2026

GitHub repository (click to open)
https://github.com/bhaveshv24/L5_53_income-tax-calculator

This document consolidates the introduction, screenshots, issue identification table, complete contents of all six reports, final verification and reflections. The corrected code and three separate tool-focused commits are pushed to the repository.

1. Introduction

Static code analysis examines source code without running the program to identify potential defects, insecure patterns and style problems. It complements execution tests and manual review; a clean report does not prove that the program is correct.

Pylint checks code quality, unused imports and variables, naming, documentation and suspicious constructs. **Bandit** scans Python for security risks, including unsafe cryptography and XML parsing. **Flake8** combines style checks and checks for common code errors to enforce readable, consistent Python.

Scope: tax_calculator.py from baseline commit 638c236. Analysis was performed in Python 3.9.6 using Pylint 3.3.9, Bandit 1.8.6 and Flake8 7.3.0. The calculator retains the repository's simplified teaching tax model.

Check	Before	After
Pylint	4.57/10	10.00/10
Bandit	6 findings (2 high, 3 medium, 1 low)	0 findings
Flake8	Violations recorded in report	0 violations
Regression tests	Not supplied	6 tests passed

2. Git repository and three commits

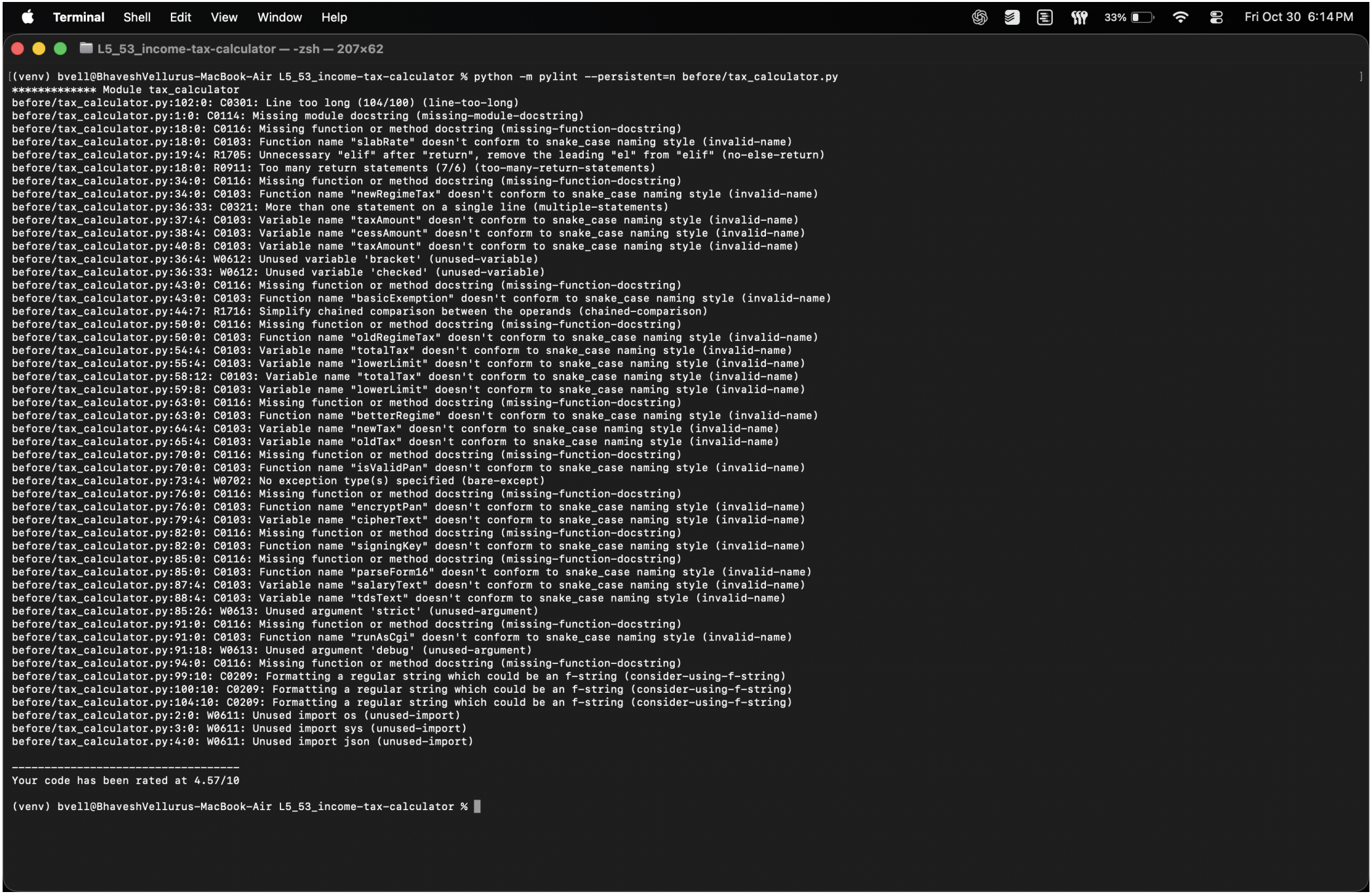
b103a81 - Bandit: security fixes and baseline reports
View commit on GitHub

ed7d923 - Pylint: quality improvements and calculation fixes
View commit on GitHub

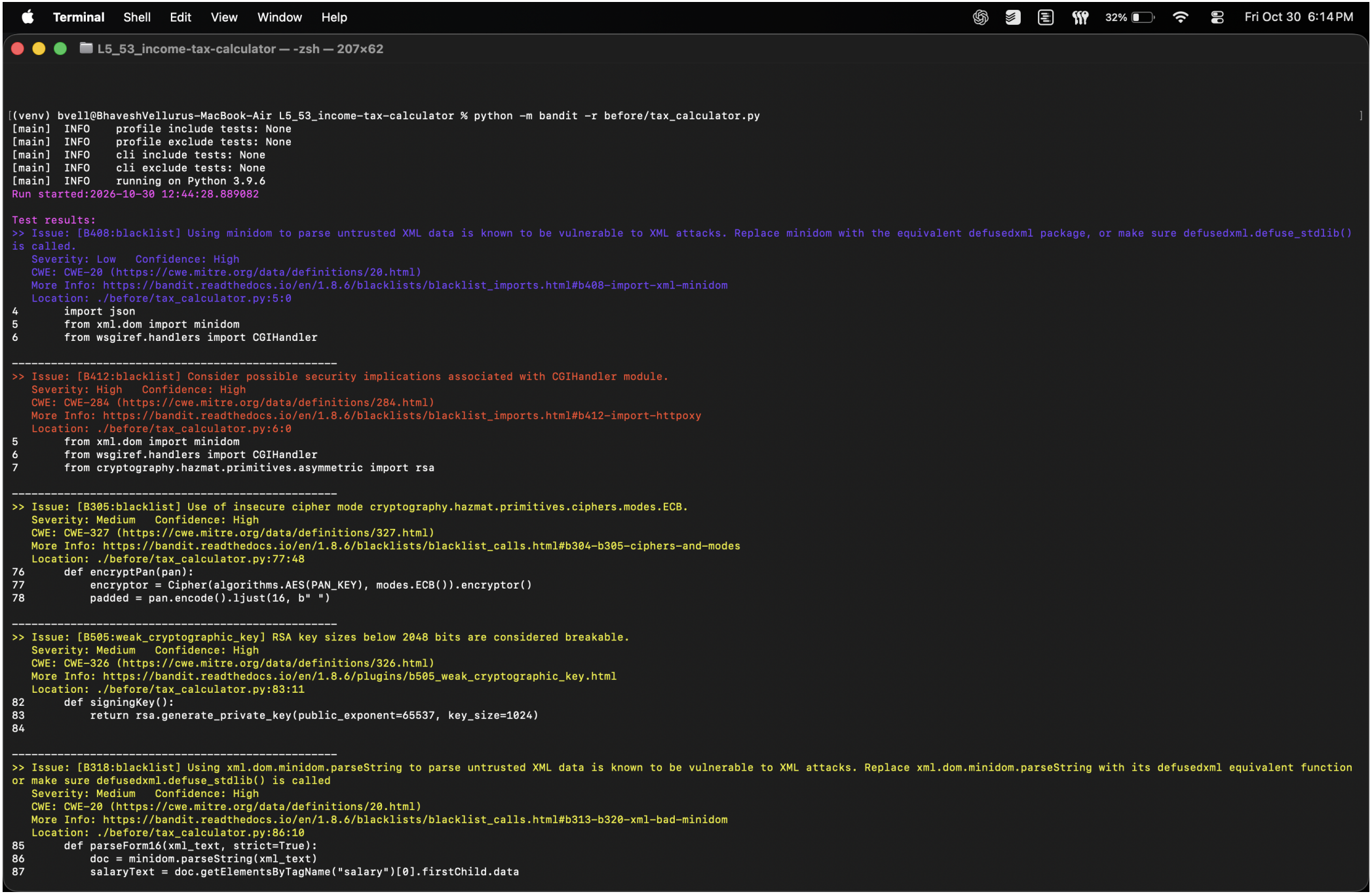
24d7fe3 - Flake8: formatting and final lab deliverables
View commit on GitHub

Exactly three commits were added after the original commit 638c236. Final remote main: 24d7fe321ad727534e0aea3a755f1537309683be. The repository includes unchanged source and reports in before/, final source and reports in after/, issues.md, reflection.md, requirements.txt and regression tests.

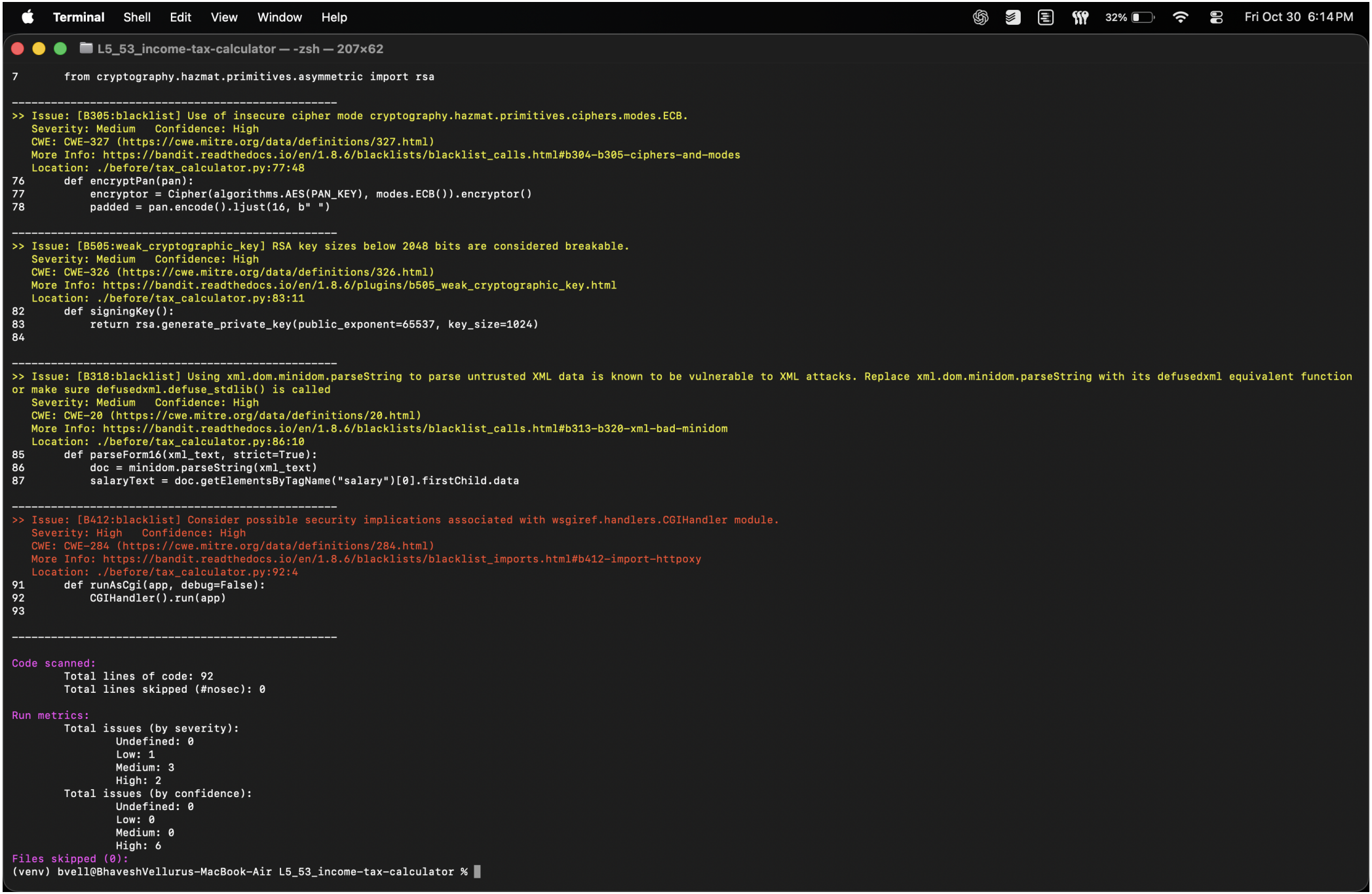
3. Before fixing: Pylint



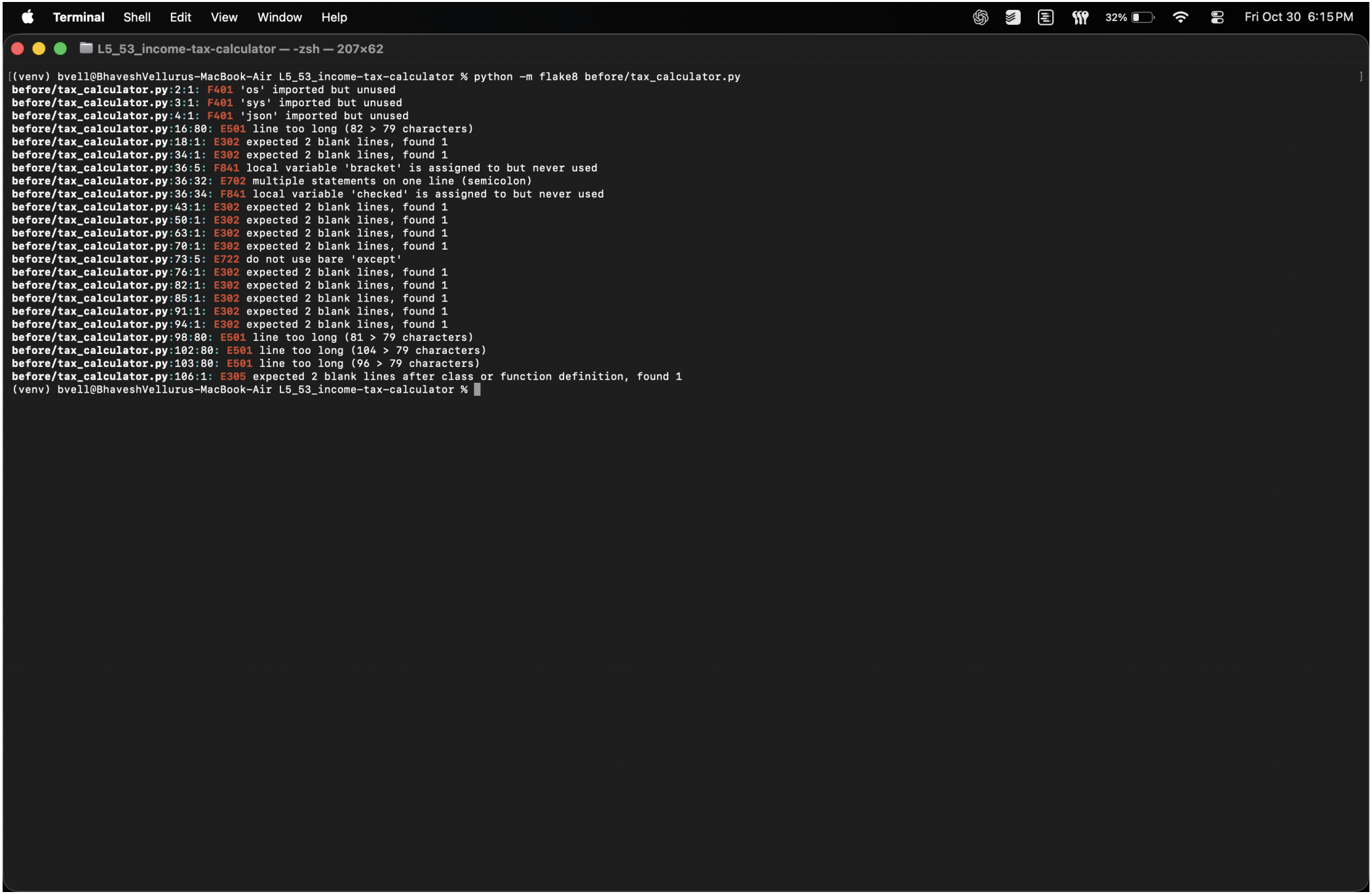
3. Before fixing: Bandit (1 of 2)



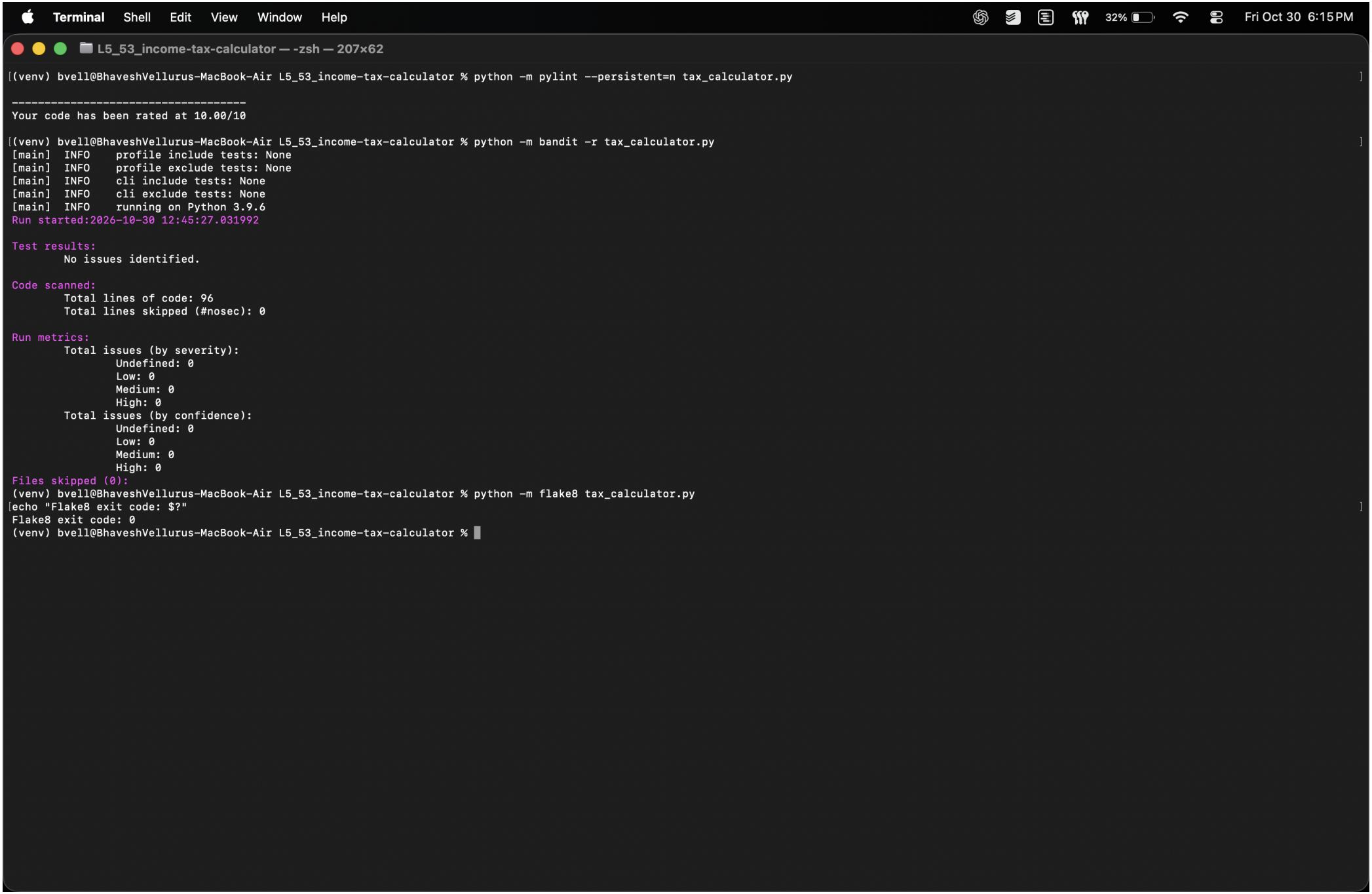
3. Before fixing: Bandit (2 of 2)



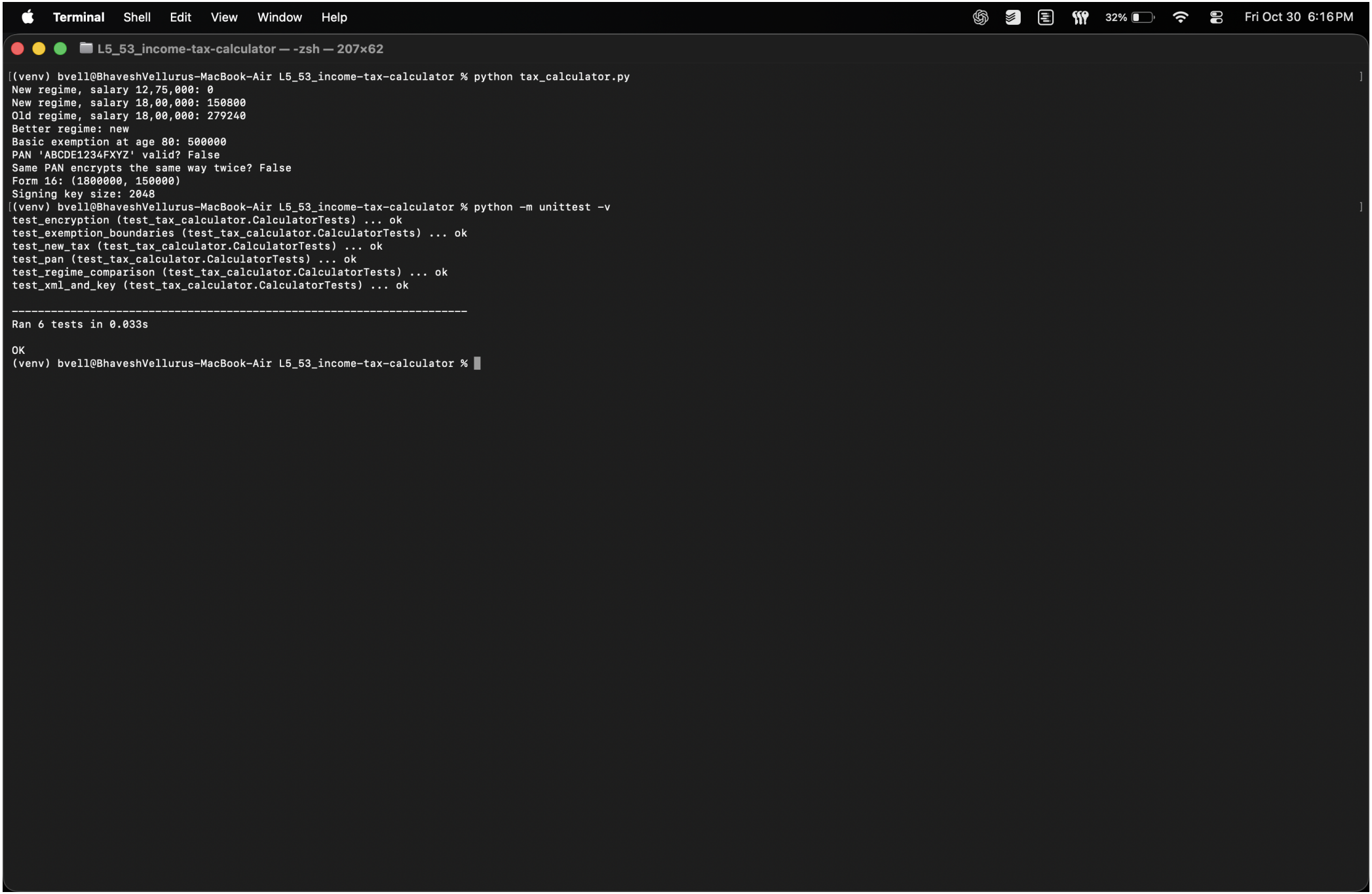
3. Before fixing: Flake8



4. After fixing: Pylint, Bandit and Flake8



4. Corrected program and regression tests



5. Issue identification table

Line numbers refer to before/tax_calculator.py. Security severity is from Bandit. Other tools use quality/style categories, not security severity. High and medium security findings are listed first.

Issue	Type	Line(s)	Description	Fix Approach
Bandit B412	High security	6, 92	CGIHandler exposes the legacy CGI/httpoxy attack surface.	Remove the unused CGI adapter and import. This teaching program remains a CLI; a web deployment would need a maintained server adapter.
Bandit B305	Medium security	77	ECB reveals repeated plaintext patterns and lacks authentication.	Use AES-GCM with a fresh 12-byte random nonce, returning nonce plus authenticated ciphertext in hex.
Bandit B505	Medium security	83	A 1024-bit RSA key is too weak.	Generate 2048-bit keys.
Bandit B318	Medium security	86	Untrusted XML parser can process malicious entity definitions.	Use defusedxml.minidom and verify entity expansion is rejected.
Bandit B408	Low security	5	Insecure standard-library XML parser import.	Replace with defusedxml.
Pylint W0702 / Flake8 E722	Exception handling	71-74	Bare except hides unrelated failures.	Check input type explicitly and remove the broad exception.
Pylint W0612 / Flake8 F841	Quality	36	bracket and checked are unused.	Remove both while rewriting progressive tax calculation.
Pylint W0611 / Flake8 F401	Quality	2-4	Unused imports obscure dependencies.	Remove sys/json; os is now used for secure nonces.
Pylint W0613	Quality	85, 91	strict/debug arguments have no effect.	Remove strict and the unused CGI helper.
Pylint C0103, C0114, C0116	Maintainability	Through out	Camel-case names and missing documentation.	Use snake_case and add module/function docstrings.
Pylint R1705, R0911, R1716	Simplification	18-32, 44	Repetitive slab branching and verbose comparison.	Replace slab lookup with progressive table traversal; use chained age comparison.
Flake8 E302, E305, E501, E702 / Pylint C0301, C0321	Style	Through out; 36, 102-103	Missing blank lines, overlong lines, semicolon statements.	Apply PEP 8 spacing and wrap expressions; remove compound statement.
Pylint C0209	Style	99-100, 104	Old-style string formatting.	Use f-strings.
Manual review	Security design	15, 76-80	Embedded key and deterministic encryption.	Require a caller-supplied key; generate a temporary demo key, with no embedded production secret.
Manual review	Calculation	35-41	Whole income taxed at a single rate; rebate excludes boundary and cess survives rebate.	Apply progressive configured slabs, inclusive rebate, then cess. Clamp taxable income at zero.

Issue	Type	Line(s)	Description	Fix Approach
Manual review	Boundary bug	46	Age exactly 80 gets the lowest exemption.	Use age >= 80 and test 59/60/79/80.
Manual review	Logic	66	Recommendation selects the more expensive regime.	Select the smaller tax; choose new on a tie.
Manual review	Validation	72	PAN prefix accepted with trailing characters.	Require re.fullmatch and reject nonstrings.
Manual review	Import side effect	106	Importing executes the demonstration.	Add a main guard.

6. Complete report: Pylint (before)

File: before/pylint_report.txt
Command: python -m pylint --persistent=n tax_calculator.py

```
************* Module tax_calculator
tax_calculator.py:102:0: C0301: Line too long (104/100) (line-too-long)
tax_calculator.py:1:0: C0114: Missing module docstring (missing-module-docstring)
tax_calculator.py:18:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:18:0: C0103: Function name "slabRate" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:19:4: R1705: Unnecessary "elif" after "return", remove the leading "el" from "elif"
(no-else-return)
tax_calculator.py:18:0: R0911: Too many return statements (7/6) (too-many-return-statements)
tax_calculator.py:34:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:34:0: C0103: Function name "newRegimeTax" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:36:33: C0321: More than one statement on a single line (multiple-statements)
tax_calculator.py:37:4: C0103: Variable name "taxAmount" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:38:4: C0103: Variable name "cessAmount" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:40:8: C0103: Variable name "taxAmount" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:36:4: W0612: Unused variable 'bracket' (unused-variable)
tax_calculator.py:36:33: W0612: Unused variable 'checked' (unused-variable)
tax_calculator.py:43:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:43:0: C0103: Function name "basicExemption" doesn't conform to snake_case naming
style (invalid-name)
tax_calculator.py:44:7: R1716: Simplify chained comparison between the operands (chained-comparison)
tax_calculator.py:50:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:50:0: C0103: Function name "oldRegimeTax" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:54:4: C0103: Variable name "totalTax" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:55:4: C0103: Variable name "lowerLimit" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:58:12: C0103: Variable name "totalTax" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:59:8: C0103: Variable name "lowerLimit" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:63:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:63:0: C0103: Function name "betterRegime" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:64:4: C0103: Variable name "newTax" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:65:4: C0103: Variable name "oldTax" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:70:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:70:0: C0103: Function name "isValidPan" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:73:4: W0702: No exception type(s) specified (bare-except)
tax_calculator.py:76:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:76:0: C0103: Function name "encryptPan" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:79:4: C0103: Variable name "cipherText" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:82:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:82:0: C0103: Function name "signingKey" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:85:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:85:0: C0103: Function name "parseForm16" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:87:4: C0103: Variable name "salaryText" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:88:4: C0103: Variable name "tdsText" doesn't conform to snake_case naming style
(invalid-name)
tax_calculator.py:85:26: W0613: Unused argument 'strict' (unused-argument)
tax_calculator.py:91:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:91:0: C0103: Function name "runAsCgi" doesn't conform to snake_case naming style
```

```
(invalid-name)
tax_calculator.py:91:18: W0613: Unused argument 'debug' (unused-argument)
tax_calculator.py:94:0: C0116: Missing function or method docstring (missing-function-docstring)
tax_calculator.py:99:10: C0209: Formatting a regular string which could be an f-string (consider-using-
f-string)
tax_calculator.py:100:10: C0209: Formatting a regular string which could be an f-string (consider-
using-f-string)
tax_calculator.py:104:10: C0209: Formatting a regular string which could be an f-string (consider-
using-f-string)
tax_calculator.py:2:0: W0611: Unused import os (unused-import)
tax_calculator.py:3:0: W0611: Unused import sys (unused-import)
tax_calculator.py:4:0: W0611: Unused import json (unused-import)

-----------------------------------
Your code has been rated at 4.57/10
```

6. Complete report: Bandit (before)

File: before/bandit_report.txt
Command: python -m bandit -r tax_calculator.py

```
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.9.6
Run started:2026-10-28 06:50:41.163083

Test results:
>> Issue: [B408:blacklist] Using minidom to parse untrusted XML data is known to be vulnerable to XML
attacks. Replace minidom with the equivalent defusedxml package, or make sure
defusedxml.defuse_stdlib() is called.
   Severity: Low   Confidence: High
   CWE: CWE-20 (https://cwe.mitre.org/data/definitions/20.html)
   More Info: https://bandit.readthedocs.io/en/1.8.6/blacklists/blacklist_imports.html#b408-import-xml-
minidom
   Location: ./tax_calculator.py:5:0
4   import json
5   from xml.dom import minidom
6   from wsgiref.handlers import CGIHandler

--------------------------------------------------
>> Issue: [B412:blacklist] Consider possible security implications associated with CGIHandler module.
   Severity: High   Confidence: High
   CWE: CWE-284 (https://cwe.mitre.org/data/definitions/284.html)
   More Info: https://bandit.readthedocs.io/en/1.8.6/blacklists/blacklist_imports.html#b412-import-
httpoxy
   Location: ./tax_calculator.py:6:0
5   from xml.dom import minidom
6   from wsgiref.handlers import CGIHandler
7   from cryptography.hazmat.primitives.asymmetric import rsa

--------------------------------------------------
>> Issue: [B305:blacklist] Use of insecure cipher mode
cryptography.hazmat.primitives.ciphers.modes.ECB.
   Severity: Medium   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.8.6/blacklists/blacklist_calls.html#b304-b305-ciphers-
and-modes
   Location: ./tax_calculator.py:77:48
76  def encryptPan(pan):
77      encryptor = Cipher(algorithms.AES(PAN_KEY), modes.ECB()).encryptor()
78      padded = pan.encode().ljust(16, b" ")

--------------------------------------------------
>> Issue: [B505:weak_cryptographic_key] RSA key sizes below 2048 bits are considered breakable.
   Severity: Medium   Confidence: High
   CWE: CWE-326 (https://cwe.mitre.org/data/definitions/326.html)
   More Info: https://bandit.readthedocs.io/en/1.8.6/plugins/b505_weak_cryptographic_key.html
   Location: ./tax_calculator.py:83:11
82  def signingKey():
83      return rsa.generate_private_key(public_exponent=65537, key_size=1024)
84

--------------------------------------------------
>> Issue: [B318:blacklist] Using xml.dom.minidom.parseString to parse untrusted XML data is known to be
 vulnerable to XML attacks. Replace xml.dom.minidom.parseString with its defusedxml equivalent function
 or make sure defusedxml.defuse_stdlib() is called
   Severity: Medium   Confidence: High
   CWE: CWE-20 (https://cwe.mitre.org/data/definitions/20.html)
   More Info: https://bandit.readthedocs.io/en/1.8.6/blacklists/blacklist_calls.html#b313-b320-xml-bad-
minidom
   Location: ./tax_calculator.py:86:10
85  def parseForm16(xml_text, strict=True):
86      doc = minidom.parseString(xml_text)
87      salaryText = doc.getElementsByTagName("salary")[0].firstChild.data
```

```
--------------------------------------------------
>> Issue: [B412:blacklist] Consider possible security implications associated with
wsgiref.handlers.CGIHandler module.
   Severity: High   Confidence: High
   CWE: CWE-284 (https://cwe.mitre.org/data/definitions/284.html)
   More Info: https://bandit.readthedocs.io/en/1.8.6/blacklists/blacklist_imports.html#b412-import-
httpoxy
   Location: ./tax_calculator.py:92:4
91  def runAsCgi(app, debug=False):
92      CGIHandler().run(app)
93

--------------------------------------------------

Code scanned:
    Total lines of code: 92
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 1
        Medium: 3
        High: 2
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 6
Files skipped (0):
```

6. Complete report: Flake8 (before)

File: before/flake8_report.txt
Command: python -m flake8 tax_calculator.py

```
tax_calculator.py:2:1: F401 'os' imported but unused
tax_calculator.py:3:1: F401 'sys' imported but unused
tax_calculator.py:4:1: F401 'json' imported but unused
tax_calculator.py:16:80: E501 line too long (82 > 79 characters)
tax_calculator.py:18:1: E302 expected 2 blank lines, found 1
tax_calculator.py:34:1: E302 expected 2 blank lines, found 1
tax_calculator.py:36:5: F841 local variable 'bracket' is assigned to but never used
tax_calculator.py:36:32: E702 multiple statements on one line (semicolon)
tax_calculator.py:36:34: F841 local variable 'checked' is assigned to but never used
tax_calculator.py:43:1: E302 expected 2 blank lines, found 1
tax_calculator.py:50:1: E302 expected 2 blank lines, found 1
tax_calculator.py:63:1: E302 expected 2 blank lines, found 1
tax_calculator.py:70:1: E302 expected 2 blank lines, found 1
tax_calculator.py:73:5: E722 do not use bare 'except'
tax_calculator.py:76:1: E302 expected 2 blank lines, found 1
tax_calculator.py:82:1: E302 expected 2 blank lines, found 1
tax_calculator.py:85:1: E302 expected 2 blank lines, found 1
tax_calculator.py:91:1: E302 expected 2 blank lines, found 1
tax_calculator.py:94:1: E302 expected 2 blank lines, found 1
tax_calculator.py:98:80: E501 line too long (81 > 79 characters)
tax_calculator.py:102:80: E501 line too long (104 > 79 characters)
tax_calculator.py:103:80: E501 line too long (96 > 79 characters)
tax_calculator.py:106:1: E305 expected 2 blank lines after class or function definition, found 1
```

6. Complete report: Pylint (after)

File: after/pylint_report.txt
Command: python -m pylint --persistent=n tax_calculator.py

```
-----------------------------------
Your code has been rated at 10.00/10
```

6. Complete report: Bandit (after)

File: after/bandit_report.txt
Command: python -m bandit -r tax_calculator.py

```
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.9.6
Run started:2026-10-28 06:51:24.278474

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 96
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 0
        High: 0
Files skipped (0):
```

6. Complete report: Flake8 (after)

File: after/flake8_report.txt
Command: python -m flake8 tax_calculator.py

The report contains zero bytes. Flake8 produced no output and exited with code 0, indicating no violations. This explanatory note is not part of the report file.

7. Behavior verification

All six regression tests pass. Checks cover configured progressive tax and rebate behavior, age boundaries, regime comparison, complete PAN matching, AES-GCM round trip and tamper rejection, XML entity rejection and 2048-bit RSA generation.

```
test_encryption (test_tax_calculator.CalculatorTests) ... ok
test_exemption_boundaries (test_tax_calculator.CalculatorTests) ... ok
test_new_tax (test_tax_calculator.CalculatorTests) ... ok
test_pan (test_tax_calculator.CalculatorTests) ... ok
test_regime_comparison (test_tax_calculator.CalculatorTests) ... ok
test_xml_and_key (test_tax_calculator.CalculatorTests) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.072s

OK
```

Original program output

```
New regime, salary 12,75,000: 124800
New regime, salary 18,00,000: 358800
Last slab rate applied: 0.3
Old regime, salary 18,00,000: 279240
Last slab rate applied: 0.3
Better regime: new
Basic exemption at age 80: 250000
PAN 'ABCDE1234FXYZ' valid? True
Same PAN encrypts the same way twice? True
Form 16: (1800000, 150000)
Signing key size: 1024
```

Corrected program output

```
New regime, salary 12,75,000: 0
New regime, salary 18,00,000: 150800
Old regime, salary 18,00,000: 279240
Better regime: new
Basic exemption at age 80: 500000
PAN 'ABCDE1234FXYZ' valid? False
Same PAN encrypts the same way twice? False
Form 16: (1800000, 150000)
Signing key size: 2048
```

8. Reflections

The repository asks for four reflection answers, but the supplied handout does not contain the question text. The following four topic-based reflections address the stated learning outcomes and are not presented as verbatim handout questions.

1. What did each analysis tool contribute?

Pylint identified maintainability problems, unused values, broad exception handling and naming/documentation issues. Flake8 highlighted layout, whitespace, long lines, unused imports and the bare except. Bandit identified six security findings involving CGIHandler, ECB encryption, weak RSA and unsafe XML. Their overlap was useful, but each tool examined different aspects of the same script.

2. How were the findings prioritized and addressed?

The high-severity CGI findings came first, followed by medium-severity ECB, RSA and XML findings. I removed the unused CGI path, used authenticated AES-GCM with random nonces and a supplied key, increased RSA to 2048 bits, and replaced the XML parser with defusedxml. Then I resolved quality and style findings. Baseline reports were preserved so the final reports can be compared with the exact original file.

3. What did static analysis miss, and how were fixes verified?

The tools did not establish that the tax calculation or recommendation was correct. Manual review found a flat-rate calculation instead of progressive slabs, a rebate boundary/cess bug, the age-80 boundary, an inverted regime recommendation and partial PAN matching. Regression tests check those cases, encryption round trips and tamper detection, XML entity rejection, and RSA key size. The final Pylint score is 10.00/10; Bandit and Flake8 both report no findings. A clean scan does not prove complete correctness or security.

4. How would this change future development?

I would run all three tools early and in continuous integration, review security findings before cosmetic changes, and combine scans with focused behavior tests and code review. Clear names, docstrings and smaller responsibilities make later changes easier to assess. Secrets should come from caller-controlled storage rather than source code. For a real tax product, I would also validate legal rules and unsupported inputs separately; this lab deliberately retains a simplified teaching model.